

Crypto Guide

Mechanisms and security contracts for shielded payments

m@rek.onl

Abstract

This volume of *The Zcash Arboretum* constructs the cryptographic mechanisms needed to understand private payments: how information is concealed, who can approve a transfer, and how its recipient recovers it. It assumes the arithmetic, encodings, probability and algebra developed in the *Math Guide*. Each security claim states what an attacker sees, what it may attempt, and which assumptions support the guarantee. Specifying a reproducible calculation is distinct from showing that it protects the information or authority entrusted to it.

Contents

1	Security contracts and reductions	4
1.1	Experiments and advantage	4
1.2	Reductions and hybrid changes	4
1.3	Group hardness contracts	5
1.4	Hash and function contracts	5
1.5	Random oracles and domain separation	6
2	Hash functions and key derivation	6
2.1	SHA-256: words, padding and compression	6
2.2	Personalised BLAKE2b	7
2.3	Expansion from a secret seed	8
2.4	Poseidon’s exact three-word permutation	8
2.5	Hashing to the Pasta groups	9
3	Groups, commitments, and Sinsemilla	11
3.1	Commitment syntax and security games	12
3.2	Pedersen commitments and their reductions	12
3.3	Adding commitments and retaining integer meaning	13
3.4	The exact Sinsemilla construction	13
3.5	Why the reused table still binds a fixed-length message	14
3.6	Exceptional inputs and the relation they expose	14
3.7	Sinsemilla commitments and shortening	15
4	Knowledge, simulation, and Schnorr	15
4.1	Completeness, soundness and knowledge	15
4.2	Simulation and what remains public	16
4.3	A three-message proof of a discrete logarithm	16
4.4	The general three-message extraction calculation	16
4.5	Fiat-Shamir and the forking lemma	17
4.6	Proof-system interfaces and explicit model boundaries	18
5	RedPallas signatures	18
5.1	Exact keys, signatures and verification	18
5.2	Unforgeability and the simulation it needs	19
5.3	Randomised signing keys	20
5.4	What a binding signature contributes to balance	20
6	Key agreement and note encryption	21
6.1	Diffie–Hellman and diversified bases	21
6.2	Authenticated encryption and nonce discipline	21
6.3	The ChaCha20 stream, word by word	22
6.4	Poly1305 and the authenticated byte string	22
6.5	What a hashed-DH confidentiality reduction proves	23
7	Authenticated trees and append-only histories	23
7.1	Binary trees with unambiguous node encodings	23
7.2	Fixed-depth append frontiers	24
7.3	Peaks, history commitments and structural binding	25

8	Wallet permutations and address encodings	25
8.1	AES-256 from the byte field	26
8.2	The exact 88-bit FF1 permutation	27
8.3	F4Jumble: a public reversible mixing function	28
8.4	Bech32m: symbols, checksum and canonical decoding	28
8.5	The composition boundary	29

1 Security contracts and reductions

An algorithm specifies outputs. A security contract specifies what an adversary sees, what it may ask, and which event counts as a violation. The same algorithm can satisfy one contract and fail another. We use the probability, conditioning and computational conventions of the *Math Guide*, “Probability and computation”, throughout.

1.1 Experiments and advantage

A digital signature scheme has three algorithms. Key generation samples a secret signing key and a corresponding public verifying key. Signing takes the secret key and a message and returns a signature, possibly using fresh randomness. Verification takes the public key, message and signature and returns accept or reject. Correctness requires accepting signatures generated honestly under that key. A signature authenticates bytes relative to a public key; it does not establish the identity of that key’s owner by itself.

A security experiment is an algorithm that samples keys and other randomness, interacts with an adversary, and returns a bit. An oracle is a callable procedure within that experiment; it may retain state between calls. A signing oracle, for example, accepts chosen messages and returns signatures under a key held by the experiment. The adversary’s next request may depend on every earlier response. Its view includes all public inputs and observed messages, together with its own internal randomness. A hidden key is not part of that view unless the experiment explicitly reveals it.

Existential unforgeability under chosen-message attack, abbreviated EUF-CMA, means that an efficient adversary given an honestly generated verifying key and a signing oracle cannot produce an accepted signature on a message it never asked the oracle to sign, except with negligible probability. Strong unforgeability, SUF-CMA, instead forbids any new accepted message–signature pair, including a new signature for an already signed message. Replaying an exact oracle response wins neither game. These definitions concern honestly generated keys; related-key and adversarial-key variants must separately specify how keys may be selected or modified.

For two experiments G_0, G_1 , the distinguishing advantage of an adversary A is

$$|\Pr[G_0^A = 1] - \Pr[G_1^A = 1]|.$$

If a uniformly sampled hidden bit chooses the experiment and the adversary guesses that bit, the absolute excess of its success probability over one half is half this distinguishing advantage, after choosing the output orientation appropriately. We keep these normalisations distinct. For search problems, such as finding a collision, advantage usually means the success probability itself.

An asymptotic statement quantifies over efficient adversaries and requires negligible advantage as the security parameter grows. A concrete statement instead gives a bound in terms of time, query counts and the chosen modulus or output length. Neither format removes the need to state the experiment. In particular, a theorem for a uniformly generated key does not automatically cover an adversarially chosen key, a key revealed after signing, or several related keys used simultaneously.

1.2 Reductions and hybrid changes

A reduction turns an adversary for one task into an algorithm for another. It receives a challenge for the second task, constructs an environment for the adversary, and transforms a successful attack into a solution to its challenge. A proved inequality of the form

$$\Pr[B \text{ solves its challenge}] \geq \frac{\text{Adv}(A) - \delta}{L}$$

records both a loss factor $L > 0$ and additive simulation error δ . The reduction’s running time and oracle access also belong to the statement. Large losses can make an asymptotic theorem weak at fixed parameters.

Often a proof replaces one part of an experiment at a time. For experiments $G_0, \dots, G_k$, the triangle inequality gives

$$|\Pr[G_0 = 1] - \Pr[G_k = 1]| \leq \sum_{i=0}^{k-1} |\Pr[G_i = 1] - \Pr[G_{i+1} = 1]|.$$

Each difference must be justified: perhaps by a computational assumption, perhaps by exact equality of distributions, or by a coupling that fails only on a bounded bad event. We use the *Math Guide*, “Probability and computation”, for these coupling and data-processing facts. Conditioning on no failure is not a substitute for proving the coupling.

1.3 Group hardness contracts

Let $\mathbb{G}$ be a prime-order group of order r with generator G , using the operations of the *Math Guide*, “Groups and scalar multiplication”. The following experiments sample scalars uniformly from $\mathbb{F}_r$; any restriction to nonzero scalars must be stated as a variant.

Assumption 1.1 (Discrete logarithm). Given G and $X = [x]G$ for uniform x , an efficient adversary has only negligible probability of returning x .

Assumption 1.2 (Computational Diffie–Hellman). Given $G, [a]G, [b]G$ for independent uniform a, b , an efficient adversary has only negligible probability of returning $[ab]G$.

Assumption 1.3 (Decisional Diffie–Hellman). The distributions $(G, [a]G, [b]G, [ab]G)$ and $(G, [a]G, [b]G, [c]G)$, with independent uniform a, b, c , are computationally indistinguishable.

These assumptions have different content. A logarithm solver computes a and hence solves the computational problem. A computational solver distinguishes the two decisional distributions by comparing its output with the last point, losing at most the accidental equality probability $1/r$ in the random case. Therefore decisional hardness is the stronger assumption; logarithm hardness alone does not imply it. Scalar zero creates explicitly easy cases of probability on the order of $1/r$, not a contradiction of an asymptotic assumption.

The curve arithmetic and parameter facts in the *Math Guide*, “Pasta curves and their encodings”, define concrete instances for these assumptions. They do not prove them. A generic search can split the unknown scalar as $x = i + jm$, with $m = \lceil \sqrt{r} \rceil$, store the m points $[i]G$, and compare against $X - [jm]G$. Some pair matches, recovering $x \bmod r$ in $O(\sqrt{r})$ group work and storage. This is an upper bound on an attack’s cost, not a lower bound for every algorithm exploiting a curve’s representation.

1.4 Hash and function contracts

A hash function has arbitrary or specified finite-length input and a fixed-length output. Collision resistance means that, after any public parameters are given, it is difficult to find distinct allowed inputs m, m' with $H(m) = H(m')$. Preimage resistance instead asks for an input matching a supplied target output. Neither property says that a secret-keyed use of the hash is pseudorandom, nor that outputs on related inputs are independent.

A pseudorandom function, or PRF, is a keyed family $F_k : D \rightarrow R$. For a randomly sampled secret key, oracle access to F_k must be indistinguishable from access to a uniformly random function $D \rightarrow R$. Repeated inputs to either oracle return the same output. The adversary may choose its inputs adaptively, subject to the contract’s query bound. If keys have a specified nonuniform distribution or

are accompanied by correlated public information, that distribution and information belong in the assumption.

A pseudorandom permutation, or PRP, is a keyed family of bijections on a finite set, compared against a uniformly random bijection. The contract specifies whether the adversary may query only the forward direction or both directions. A random permutation never collides on distinct inputs; a random function may. The distinction is material when query counts approach the square root of the domain size. Bijectivity alone is not a PRP proof.

1.5 Random oracles and domain separation

In the random-oracle model, all parties can query the same ideal random function. A lazy implementation maintains a table. It answers an old input from the table; for a new input it samples a uniform output, stores it, and returns it. This describes exactly the finite transcript distribution of a random function without having to sample an infinite table in advance.

A proof may simulate that oracle, including programming a chosen output at a fresh input. Programming an already answered input would make the transcript inconsistent. Even at a fresh input, the programmed value and the rest of the simulated transcript must have the required joint distribution. A real hash algorithm is a fixed public computation, not a programmable oracle; a random-oracle theorem is a model-dependent security statement about its idealisation.

If domains are encoded injectively as $\text{enc}(D, m)$, then disjoint domain labels query disjoint inputs to the ideal oracle. Their restrictions are therefore independent random functions. Ambiguous concatenation can destroy this reasoning. We use the tuple and byte conventions of the *Math Guide*, “Language, integers, and encodings”. A personalisation parameter inside a concrete hash is likewise part of its algorithm; modelling different personalisations as independent functions is an additional security idealisation, not an algebraic consequence of changing a few initial-state bits.

2 Hash functions and key derivation

The constructions in this section are executable specifications. Their security contracts are identified separately. Byte-level hashes operate on word arithmetic and bit operations from the *Math Guide*, “Language, integers, and encodings”; the group maps use “Pasta curves and their encodings”; the field permutation uses “Prime fields and modular arithmetic” and “Linear algebra and uniform masks”.

2.1 SHA-256: words, padding and compression

SHA-256 maps byte strings shorter than 2^{61} bytes to 32 bytes. All its additions are modulo 2^{32} . Let $\text{rotr}_j(x)$ be right rotation of a 32-bit word, and $\text{shr}_j(x) = \lfloor x/2^j \rfloor$. Define

$$\begin{aligned} \text{Ch}(x, y, z) &= (x \& y) \oplus ((\neg x) \& z), \\ \text{Maj}(x, y, z) &= (x \& y) \oplus (x \& z) \oplus (y \& z), \\ \Sigma_0(x) &= \text{rotr}_2(x) \oplus \text{rotr}_{13}(x) \oplus \text{rotr}_{22}(x), \\ \Sigma_1(x) &= \text{rotr}_6(x) \oplus \text{rotr}_{11}(x) \oplus \text{rotr}_{25}(x), \\ \sigma_0(x) &= \text{rotr}_7(x) \oplus \text{rotr}_{18}(x) \oplus \text{shr}_3(x), \\ \sigma_1(x) &= \text{rotr}_{17}(x) \oplus \text{rotr}_{19}(x) \oplus \text{shr}_{10}(x). \end{aligned}$$

The operations $\&$ and $\neg$ are bitwise and and 32-bit complement. Initialise eight state words to $I_j = \lfloor 2^{32} \sqrt[p_j]{} \rfloor \bmod 2^{32}$, where p_j , indexed from zero, are the first eight primes. Define 64 round constants $K_j = \lfloor 2^{32} \sqrt[3]{p_j} \rfloor \bmod 2^{32}$ using the first 64 primes. These are exact integer constructions: compute the integer square root of $p_j 2^{64}$, or integer cube root of $p_j 2^{96}$, respectively, then retain the low 32 bits. An integer d -th root is the largest integer u with $u^d \leq n$, found by binary search; no floating-point rounding is required. Trial division generates the required finite prime list, beginning 2, 3, 5, 7, 11, 13, 17, 19.

For a message of bit length L , append one bit, then enough zero bits to make the length 448 modulo 512, then append the 64-bit big-endian encoding of L . Parse each resulting 64-byte block into 16 big-endian words $W_0, \dots, W_{15}$. Extend them for $16 \leq j < 64$ by

$$W_j = \sigma_1(W_{j-2}) + W_{j-7} + \sigma_0(W_{j-15}) + W_{j-16}.$$

Copy the current state into (a, b, c, d, e, f, g, h) . For each $j = 0, \dots, 63$, calculate

$$T_1 = h + \Sigma_1(e) + \text{Ch}(e, f, g) + K_j + W_j, \quad T_2 = \Sigma_0(a) + \text{Maj}(a, b, c),$$

then simultaneously replace the working words by $(T_1 + T_2, a, b, c, d + T_1, e, f, g)$. Add the eight final working words to the eight chaining-state words. Repeat for every block, then concatenate the eight state words in big-endian order.

This constructs the SHA-256 algorithm in FIPS 180-4, sections 5 and 6.2. Applying it twice defines $\text{SHA256d}(m) = \text{SHA256}(\text{SHA256}(m))$. Neither double application nor the padding structure is, by itself, a proof of collision resistance or a random-oracle property. Where a protocol uses a tagged wrapper, its precise prefix is an additional construction, not a different SHA-256 compression function.

2.2 Personalised BLAKE2b

We need the sequential, unkeyed BLAKE2b function with an output length d between 1 and 64 bytes and a 16-byte personalisation P . The adjective unkeyed refers to BLAKE2b's native key parameter; a secret may still occur in the message of a higher-level PRF construction. The following fixed initialisation specifies the unkeyed sequential mode; no optional modes are used.

All additions now use 64-bit words. Define $I_j = \lfloor 2^{64} \sqrt{p_j} \rfloor \bmod 2^{64}$ for the same first eight primes, constructible as integer square roots of $p_j 2^{128}$. Initialise $h_j = I_j$, then apply

$$h_0 \leftarrow h_0 \oplus (0x01010000 \oplus d), \quad h_6 \leftarrow h_6 \oplus \text{val}_{256}^{\text{LE}}(P[0..8]), \quad h_7 \leftarrow h_7 \oplus \text{val}_{256}^{\text{LE}}(P[8..16]).$$

Slices use an inclusive lower and exclusive upper byte index. The notation $\text{val}_{256}^{\text{LE}}$ interprets a byte slice little-endian. A shorter application label is padded with zero bytes to 16 bytes before becoming P ; longer labels require a different construction and are not silently truncated.

Split the message into 128-byte blocks, padding the final block with zeros if necessary. An empty message has one all-zero final block. A nonempty exact multiple of 128 bytes has its last existing block marked final; no extra block is appended. For each block, parse 16 little-endian 64-bit message words m_i . Let t be the number of actual message bytes processed through this block, excluding padding, represented as a 128-bit integer; the total message length must be less than 2^{128} bytes. Initialise $v_0, \dots, v_7 = h_0, \dots, h_7$ and $v_8, \dots, v_{15} = I_0, \dots, I_7$; exclusive-or the low and high 64 bits of t into v_{12}, v_{13} , and complement v_{14} for the final block.

The mixing operation $G(a, b, c, d, x, y)$ updates four selected working words in this order. All additions now use 64-bit words, and rotr_j rotates a 64-bit word right, not the 32-bit word used in SHA-256:

$$\begin{aligned} a &\leftarrow a + b + x, & d &\leftarrow \text{rotr}_{32}(d \oplus a), \\ c &\leftarrow c + d, & b &\leftarrow \text{rotr}_{24}(b \oplus c), \\ a &\leftarrow a + b + y, & d &\leftarrow \text{rotr}_{16}(d \oplus a), \\ c &\leftarrow c + d, & b &\leftarrow \text{rotr}_{63}(b \oplus c). \end{aligned}$$

For round j , use the row $\sigma_{j \bmod 10}$ from the following table. Each row lists a permutation of message-

word indices, written in hexadecimal so every index occupies one digit.

0	0123456789abcdef
1	ea489fd61c02b753
2	b8c052fdae367194
3	7931dcbe265a40f8
4	905724afe1bc683d
5	2c6a0b834d75fe19
6	c51fed4a0763928b
7	db7ec13950f4862a
8	6fe9b308c2d714a5
9	a2847615fb9e3cd0

Spaces in a row only aid reading. Apply G to working-word index quadruples

$$(0, 4, 8, 12), (1, 5, 9, 13), (2, 6, 10, 14), (3, 7, 11, 15),$$

then

$$(0, 5, 10, 15), (1, 6, 11, 12), (2, 7, 8, 13), (3, 4, 9, 14).$$

The k -th application, indexed from zero, receives message arguments $m_{\sigma(2k)}, m_{\sigma(2k+1)}$. Perform 12 rounds, then set $h_i \leftarrow h_i \oplus v_i \oplus v_{i+8}$ for $0 \leq i < 8$. After the final block, serialize the eight state words little-endian and take the first d bytes.

Write the result as $\text{BLAKE2b}_{8d}(P, m)$, where the subscript counts bits and the internal parameter d counts bytes. The output length is part of initialisation: a 32-byte output is not the first half of the 64-byte-output function. The compression is described in RFC 7693; personalisation follows the parameter block in the protocol specification’s “BLAKE2 Hash Functions” and the `blake2b_simd` implementation. All later calls fix d, P explicitly.

2.3 Expansion from a secret seed

For a 32-byte seed k and byte string t , define

$$\text{PRFexpand}_k(t) = \text{BLAKE2b}_{512}(\text{Zcash_ExpandSeed}, k || t). \quad (1)$$

The 16-byte label is exact. This is an unkeyed BLAKE2b invocation whose message begins with the secret seed, not BLAKE2b with its native key parameter set to k . The subscript on PRFexpand identifies the secret input in the PRF contract. Distinct application purposes use their specified tag bytes in t , together with any additional length-fixed fields.

For a 64-byte string b , define

$$\text{ToBase}(b) = [\text{val}_{256}^{\text{LE}}(b)]_{p_{\text{Pallas}}}, \quad \text{ToScalar}(b) = [\text{val}_{256}^{\text{LE}}(b)]_{p_{\text{Vesta}}}.$$

These are reduction functions, not canonical field decoders. A uniform 512-bit input yields the small, precisely bounded reduction bias from the *Math Guide*, “Probability and computation”. The PRF assumption for (1), including the actual seed distribution and public leakage of its outputs, is separate from that arithmetic calculation. Revealing one derived key may change which other secrecy claims a key hierarchy can support; domain separation alone does not prove every such claim.

2.4 Poseidon’s exact three-word permutation

Let $p = p_{\text{Pallas}}$. The permutation acts on $\mathbf{s} \in \mathbb{F}_p^3$. It has 64 rounds with three round constants per round and one fixed three-by-three matrix M . In rounds 0, 1, 2, 3, 60, 61, 62, 63, first add the three constants and raise all three words to the fifth power. In rounds 4 through 59, add all three constants but raise only word zero to the fifth power. End every round by replacing the state with $M\mathbf{s}$. There

are thus eight full rounds and 56 partial rounds; the latter do not omit the additions to the other two words.

The following generator fixes every constant, avoiding an undefined external table. Initialise an 80-bit register by concatenating, in most-significant-bit-first order,

$$\text{BE}_2(1) \parallel \text{BE}_4(0) \parallel \text{BE}_{12}(255) \parallel \text{BE}_{12}(3) \parallel \text{BE}_{10}(8) \parallel \text{BE}_{10}(56) \parallel 1^{30}.$$

Here BE_k denotes exactly k bits, and 1^{30} means 30 one bits. For register bits $s_0, \dots, s_{79}$, one raw step computes $b = s_0 \oplus s_{13} \oplus s_{23} \oplus s_{38} \oplus s_{51} \oplus s_{62}$, shifts the register left by one position, appends b , and outputs b . Discard 160 raw outputs. Thereafter read raw outputs in pairs: emit the second bit exactly when the first is one, otherwise discard the pair and repeat. This is a deterministic parameter generator, not a source of secret random bits.

To generate one round constant, read 255 emitted bits as a big-endian integer; reject and read another 255 bits if it is at least p . Generate 192 accepted constants and arrange them row-by-row into the 64 rounds. Continue from the resulting register state to generate six further 255-bit integers, this time reducing each modulo p without rejection. If any of the six residues coincide, discard the whole six-value candidate and continue. Call the first three x_i and the last three y_j , and set

$$M_{ij} = (x_i + y_j)^{-1}.$$

For these pinned parameters the first distinct candidate is used, and every displayed denominator is nonzero; the checker verifies this and the generated constants against the implementation. The algorithm is the parameter generation in `halo2_poseidon 0.1.0`, `src/grain.rs`, `src/mds.rs` and `src/p128pow5t3.rs`, corresponding to the protocol specification’s “PoseidonHash Function”.

For distinct x_i , distinct y_j , and nonzero sums, every square submatrix of this matrix is invertible. To prove it for a k -by- k submatrix, suppose coefficients a_j give $\sum_j a_j/(x_i + y_j) = 0$ at its k distinct row values. Multiply by $\prod_j (X + y_j)$. The resulting polynomial has degree less than k and k roots, hence is zero. Evaluating at $X = -y_j$ forces $a_j = 0$ because all other factors are nonzero. This proves the matrix property called maximum distance separable, or MDS, in this construction. It is a diffusion property, not a security proof for the permutation.

The fifth-power map is a bijection because $\gcd(5, p - 1) = 1$: an inverse exponent exists modulo $p - 1$ on nonzero elements, and zero is fixed. Constant addition and matrix multiplication are also bijections, so every round and the complete function π are permutations. The two-input hash is exactly

$$\text{PoseidonHash}(x, y) = \pi(x, y, 2^{65})_0. \quad (2)$$

The last word fixes the constant two-element input length and one-element output length; no extra input padding or permutation round is performed. The nullifier PRF contract treats its first input as a secret key and its second as the chosen argument. Permutation invertibility, the MDS proof, and reproducibility of the round constants do not establish that PRF contract. We assume the required pseudorandomness for the specified key distribution and leakage, rather than inferring it from collision resistance.

2.5 Hashing to the Pasta groups

A generator derived from a public message must be a group point, not an unchecked coordinate or a known scalar times a fixed base. We define $\text{GroupHash}_C(D, m)$ for target curve C equal to Pallas or Vesta. The base modulus p is the corresponding one from the *Math Guide*, “Pasta curves and their encodings”. The domain D and message m are bytes. Set

$$\text{DST} = D \parallel - \parallel \text{name}(C) \parallel \text{_XMD:BLAKE2b_SSWU_R0_},$$

where the names are the lowercase ASCII strings `pallas` and `vesta`. Reject if this string is longer than 255 bytes. Append its one-byte length to obtain DST' . With all-zero BLAKE2b personalisation,

$$\begin{aligned} b_0 &= \text{BLAKE2b}_{512}(0^{16}, 0^{128} \| m \| [0, 128, 0] \| \text{DST}'), \\ b_1 &= \text{BLAKE2b}_{512}(0^{16}, b_0 \| [1] \| \text{DST}'), \\ b_2 &= \text{BLAKE2b}_{512}(0^{16}, (b_0 \oplus b_1) \| [2] \| \text{DST}'). \end{aligned}$$

The intermediate curve has equation $y^2 = x^3 + Ax + B$, with $B = 1265$, $Z = -13$, and

C	A as a hexadecimal integer modulo p
Pallas	18354a2eb0ea8c9c49be2d7258370742b74134581a27a59f92bb4b0b657a014b
Vesta	267f9b2ee592271a81639c4d96f787739673928c7d01b212c515ad7242eaa6b1

The square root in the second branch exists. When $u \neq 0$ and $t^2 + t \neq 0$, substitution gives

The value t is a nonsquare because Z is a nonsquare and u^2 is a nonzero square. Multiplication by t^3 therefore exchanges square and nonsquare values. The exceptional case is also completely specified: since -1 is square in both Pasta fields whereas Z is nonsquare, $Zu^2 = -1$ is impossible. Thus $t^2 + t = 0$ requires $u = 0$, and for the stated constants $g(B/(ZA))$ is a nonzero square, as verified by Euler’s criterion in the checker. The parity selection makes the output independent of the square-root algorithm’s choice. This affine form is equivalent to the denominator-cleared SWU implementation in `pasta_curves`; it exposes the divisions and exceptional case explicitly.

$$\begin{aligned} N_x &= c_0x^3 + c_1x^2 + c_2x + c_3, & D_x &= x^2 + c_4x + c_5, \\ N_y &= (c_6x^3 + c_7x^2 + c_8x + c_9)y, & D_y &= x^3 + c_{10}x^2 + c_{11}x + c_{12}. \end{aligned}$$

Pallas:

10

[illegible]

The primary definition is the protocol specification’s “Group Hash into Pallas and Vesta”, implemented in `pasta_curves 0.5.2, src/hashtocurve.rs` and `src/curves.rs`. Its exact XMD framing takes precedence over a differently parameterised generic hash-to-curve suite. A caller that requires a nonidentity point must check or otherwise establish that requirement. For an 11-byte diversifier d , the concrete diversification wrapper is

Then

$$\text{DiversifyHash}(d) = \begin{cases} \text{GroupHash}_{\text{Pallas}}(\text{z.cash:Orchard-gd}, \epsilon), & P = \mathcal{O}, \\ P, & P \neq \mathcal{O}. \end{cases}$$

3 Groups, commitments, and Sinsemilla

11

3.1 Commitment syntax and security games

A commitment algorithm takes public parameters, a message m and randomness r , and returns C . An opening consists of the message and the auxiliary information needed to verify the commitment; for the constructions here it is (m, r) . Correctness means an honestly computed opening verifies.

In the hiding experiment, the adversary chooses two allowed messages of the same permitted type, a hidden random bit chooses one, and the adversary sees its commitment with freshly sampled randomness. Perfect hiding means the two distributions are identical; statistical hiding bounds their statistical distance; computational hiding bounds the advantage of efficient distinguishers. In the binding experiment, the adversary returns one commitment and two accepted openings with different messages. Computational binding requires its success probability to be negligible. Neither game requires an arbitrary commitment to come with an efficiently extractable opening.

3.2 Pedersen commitments and their reductions

In a prime-order group of order r , choose nonidentity points G, H . For $m, \rho \in \mathbb{F}_r$, define

$$\text{Com}(m; \rho) = [m]G + [\rho]H.$$

Verification recomputes the right side. For each fixed message, uniform ρ makes the commitment uniform on the group: multiplication by H is a bijection, and addition of $[m]G$ is a translation. This proves perfect hiding exactly, provided ρ is uniform and independent of the message.

Theorem 3.1 (Pedersen binding reduction). *Two openings $(m, \rho), (m', \rho')$ of the same commitment, with $m \neq m'$, yield the discrete logarithm of H relative to G .*

Proof. Subtract the opening equations to obtain $[m - m']G + [\rho - \rho']H = \mathcal{O}$. The scalar $\rho - \rho'$ cannot be zero, because G has order r and $m - m' \neq 0$. Writing $H = [h]G$, scalar injectivity gives $h = -(m - m')/(\rho - \rho')$ in $\mathbb{F}_r$. $\square$

The reduction assumes the public parameters have the promised distribution and that the adversary does not already know that logarithm. Choosing $H = [h]G$ with a publicly known h would destroy binding immediately, despite leaving perfect hiding intact. A public hash-to-group procedure supplies reproducible bases; its unknown-relation property uses the security model stated in §2.

The vector construction is

$$\text{Com}(\mathbf{m}; \rho) = \sum_{i=0}^{n-1} [m_i]G_i + [\rho]H.$$

The same translation argument proves perfect hiding. A pair of distinct vector openings yields a nonzero coefficient relation among the public bases. The following lemma connects that event to ordinary discrete logarithms without calling the bases linearly independent.

Lemma 3.2 (Unknown relations from discrete logarithms). *For independent uniform group bases $H_1, \dots, H_N$, an algorithm returning a nonzero coefficient vector with $\sum_i [c_i]H_i = \mathcal{O}$, with probability ϵ , gives a discrete-log solver with success probability at least $\epsilon(1 - 1/r)$. The statement also permits independently uniform nonidentity bases.*

Proof. Given $X = [x]G$, sample independent uniform scalars a_i, b_i and supply $H_i = [a_i]G + [b_i]X$. For the fixed hidden scalar x , the change of variables $(a_i, b_i) \mapsto (a_i + xb_i, b_i)$ is a bijection. Hence the bases have the required independent uniform distribution, and the hidden b_i remain uniform independent of the entire adversary view and returned relation. For nonidentity bases, resample any identity result; conditioning on the first coordinate being nonzero leaves the second uniform.

For a nonzero returned vector, the scalar $B = \sum_i c_i b_i$ is uniform, by solving for any b_i whose coefficient is nonzero. It is zero with probability $1/r$. Otherwise the relation gives $x = -(\sum_i c_i a_i)/B$, which the reduction can compute. The argument conditions on the bases and the returned relation, not on hidden coefficients unavailable to the adversary. $\square$

Random-oracle-derived bases can be simulated by this construction at each distinct domain/input pair, reusing the same sampled point on repeat queries. A concrete hash-to-curve implementation is covered only under the corresponding idealisation or an explicit imported composition result. Fixed bases that happen to be distinct are not enough for the lemma’s hypothesis.

3.3 Adding commitments and retaining integer meaning

The scalar identities imply

$$\text{Com}(m; \rho) + \text{Com}(m'; \rho') = \text{Com}(m + m'; \rho + \rho').$$

Subtraction and multiplication by public scalars obey analogous identities. This property is called additive homomorphism. It holds in $\mathbb{F}_r$, not automatically for unbounded integers. To infer an integer balance, use the *Math Guide*, “Prime fields and modular arithmetic”, with bounds on the complete signed sum. Knowing that all individual amounts fit their encodings is insufficient without a bound preventing wraparound in the aggregate.

There is also a knowledge boundary. A point always has some discrete logarithm relative to a generator; existence of that scalar does not mean a prover knows it. An application that combines a commitment-opening proof with a signature must state how the relevant witnesses can be extracted together. The binding theorem only turns two *given* inconsistent openings into a hard-problem solution.

3.4 The exact Sinsemilla construction

Use the Pallas group, with scalar modulus $r = p_{\text{Vesta}}$. The input is a domain byte string D and at most 2530 bits M . Split the input into pieces of ten bits after appending zero bits to a multiple of ten. For the empty input use no pieces. Interpret each piece little-endian as an integer $m_i \in \{0, \dots, 1023\}$. The number of pieces is $n = \lceil |M|/10 \rceil \leq 253$. Define the bases

$$\begin{aligned} Q_D &= \text{GroupHash}_{\text{Pallas}}(\text{z.cash:SinsemillaQ}, D), \\ S_j &= \text{GroupHash}_{\text{Pallas}}(\text{z.cash:SinsemillaS}, \text{LE}_4^{\text{byte}}(j)). \end{aligned}$$

The table has 1024 entries indexed by piece *value*, shared by every position. It is not a new generator for each message position. Required fixed generators are checked to be nonidentity; a general adversarially selected domain needs the corresponding exceptional-base event included in its security analysis.

Let $\boxplus$ be the incomplete addition defined in the *Math Guide*, “Pasta curves and their encodings”, extended to propagate $\perp$. Set $A_0 = Q_D$ and for $1 \leq i \leq n$ compute

$$A_i = (A_{i-1} \boxplus S_{m_i}) \boxplus A_{i-1}. \quad (3)$$

The result $\text{HashToPoint}(D, M)$ is A_n , possibly $\perp$. The coordinate hash applies the extended Extract function, preserving failure. These algorithms are specified in “Sinsemilla Hash Function” of the protocol specification and implemented by the `sinsemilla` crate’s hash-domain and incomplete-addition operations.

3.5 Why the reused table still binds a fixed-length message

When no exceptional addition occurs, the complete group law gives $A_i = [2]A_{i-1} + S_{m_i}$. Induction therefore yields

$$A_n = [2^n]Q_D + \sum_{i=1}^n [2^{n-i}]S_{m_i} = [2^n]Q_D + \sum_{j=0}^{1023} [\chi_j(m)]S_j,$$

where $\chi_j(m) = \sum_{i: m_i=j} 2^{n-i}$. The binary digits of χ_j identify exactly the positions occupied by piece value j . Since $0 \leq \chi_j < 2^n < r$, scalar reduction loses none of these digits. Thus the complete coefficient vector determines every piece. Padding is injective on inputs of any one fixed bit length, so it also determines the original message under that restriction.

Theorem 3.3 (Fixed-length Sinsemilla collision reduction). *For one domain and one fixed input length, two distinct messages with equal successful point hashes give a nonzero relation among the table bases. Equal successful coordinate hashes also give a nonzero relation, allowing the initial base Q_D .*

Proof. For point equality subtract the two coefficient expressions. The initial-base coefficients cancel, while the table coefficient difference is nonzero by injectivity. For equal coordinates, the *Math Guide*, “Pasta curves and their encodings”, shows that the points are equal or opposite, including the identity case. Equality was handled already. For opposites, add the two expressions. The coefficient of Q_D is 2^{n+1} , which is nonzero modulo r because $2^{n+1} \leq 2^{254} < r$. The relation is therefore nontrivial regardless of the table coefficients. $\square$

Apply Lemma 3.2 in the independently distributed-base model to obtain the discrete-log reduction. The fixed-length hypothesis is essential: appending zero bits without changing the number of pieces can leave the padded message unchanged. A domain label is not a length encoding unless the application assigns one fixed length to it or encodes lengths injectively itself.

3.6 Exceptional inputs and the relation they expose

Suppose the initial and table bases are nonidentity, and consider the first failing iteration. All preceding incomplete additions agree with complete addition, so $A = A_{i-1}$ has the known coefficient expression above and is nonidentity. Write $S = S_{m_i}$. The first addition fails precisely when $A = S$ or $A = -S$. If it succeeds, its output $A + S$ is nonidentity. The second addition fails when $A + S = A$ or $A + S = -A$. The first would require $S = \mathcal{O}$, excluded by hypothesis; the second gives $[2]A + S = \mathcal{O}$. Thus every first failure exposes

$$[\alpha]A + S = \mathcal{O}, \quad \alpha \in \{-1, 1, 2\}.$$

Substituting the known expression for A gives a relation whose initial-base coefficient is $\alpha 2^{i-1} \neq 0$ modulo r . Hence finding a failure efficiently also finds a nontrivial generator relation. An identity initial or table base is itself such a relation and must not be silently ignored when the model allows arbitrary domain queries.

This is a computational exceptional-input argument, not an unconditional statement that $\boxplus$ never fails. It also does not say that denominator-cleared circuit equations reject every exceptional assignment. Their exact relation must be compared with the first-exception argument; otherwise the circuit and the specified partial function could describe different behaviours on precisely the inputs omitted by the ordinary formula.

3.7 Sinsemilla commitments and shortening

For a commitment domain D , use

$$H_D = \text{GroupHash}_{\text{Pallas}}(D||-r, \epsilon), \quad T = \text{HashToPoint}(D||-M, M).$$

If $T = \perp$, return $\perp$. Otherwise define

$$\text{SinsemillaCommit}_D(M; \rho) = T + [\rho]H_D, \quad \text{ShortCommit}_D(M; \rho) = \text{Extract}(T + [\rho]H_D).$$

The final addition is *complete*. Replacing it with incomplete addition changes the function and its valid domain. For every message whose point hash succeeds, uniform independent ρ gives a uniform group commitment when $H_D \neq \mathcal{O}$; deterministic coordinate extraction preserves perfect hiding, although it need not produce a uniform field element. This is hiding over the admitted message domain, not a claim that the public failure bit hides which invalid message was attempted.

For fixed-length messages, two inconsistent point openings produce the table-coefficient relation above with an additional coefficient of H_D . Equal shortened openings either reduce to this case or have opposite underlying points, whose initial-base coefficient 2^{n+1} again prevents the relation from being trivial. Thus shortening retains computational binding under the same base model. This does not promote coordinate extraction to an injective map.

The application domain `z.cash:Orchard-NoteCommit` instantiates note commitments. The short commitment domain `z.cash:Orchard-CommitIvk` instantiates `CommitIvk` on the fixed 510-bit concatenation of two canonical 255-bit field encodings. These are applications of the exact commitment construction in the protocol specification’s “Sinsemilla Commitments”, not new group operations. The upper-level protocol supplies which values occupy those encodings, which randomness is permitted, and which exceptional or zero outputs invalidate a key or note.

4 Knowledge, simulation, and Schnorr

A relation $R \subseteq X \times W$ specifies allowed pairs of public statements x and private witnesses w , with an efficient algorithm deciding whether $R(x, w) = 1$. The statement is true if at least one such witness exists. A prover has a witness and sends messages intended to convince a verifier that the statement is true. We now distinguish the properties that such an exchange may satisfy.

4.1 Completeness, soundness and knowledge

An interactive protocol specifies the messages, random choices, state transitions and final verification rule. Completeness means that an honest prover with a valid witness is accepted, perfectly or except with a stated error probability. Soundness means that a false statement is accepted with at most a stated probability even when the prover deviates. When this restriction is proved only against computationally bounded provers, the protocol is called an argument rather than an unconditional proof. This distinction concerns the guarantee, not whether the transmitted object is colloquially called a proof.

Knowledge soundness is stronger than ordinary soundness. It requires an extractor: an algorithm that, from suitably successful interaction with the alleged prover, obtains a witness satisfying the same relation. The definition must specify the extractor’s access. It may receive the prover’s code, invoke it only through calls without inspecting its code, or restore a saved execution state while keeping every random choice fixed. The last operation is called rewinding; the sequence of fixed random choices is called its random tape. These are different models. A knowledge error κ allows success up to κ without requiring extraction; above it, the extractor’s running time is bounded in terms of the prover’s cost and its excess success probability. Existence of a witness in the mathematical relation is not knowledge of it.

Several proofs about one object may also need joint extraction. Separate statements that each accepting proof has some witness do not necessarily provide one extractor producing all consistent witnesses in the same adversarial execution. The surrounding application must state its relation and composition theorem precisely enough to make that requirement meaningful.

4.2 Simulation and what remains public

A simulator produces the verifier's view without receiving the witness. A protocol is zero knowledge when, for true statements, that simulated view matches the real view in the specified sense: exactly for perfect zero knowledge, within a small statistical distance for statistical zero knowledge, or against efficient distinguishers for computational zero knowledge. The simulator receives the public statement and whatever public or auxiliary input the definition permits. It is not required to hide information already determined by those inputs.

The view includes correlated messages, not just each message's marginal distribution. The *Math Guide*, "Linear algebra and uniform masks", exhibits two individually uniform observations that together reveal a secret combination. A simulation proof must preserve such correlations and the adversary's permitted interactions. Honest-verifier zero knowledge considers the prescribed verifier's challenge distribution; general zero knowledge permits a deviating verifier. General zero knowledge includes the honest-verifier case; the converse requires an additional argument.

4.3 A three-message proof of a discrete logarithm

Fix the group and generator of §1. The public statement is Y , and the witness is $x \in \mathbb{F}_r$ with $Y = [x]G$. The Schnorr exchange is:

1. The prover samples $k \leftarrow \mathbb{F}_r$ and sends $A = [k]G$.
2. The verifier samples $c \leftarrow \mathbb{F}_r$ and sends it.
3. The prover returns $z = k + cx$. The verifier accepts exactly when $[z]G = A + [c]Y$.

All encodings and point/scalar validity checks are part of the concrete implementation of this abstract exchange. Completeness follows directly by distributing scalar multiplication over $k + cx$.

Lemma 4.1 (Special soundness). *From two accepting transcripts (A, c, z) , (A, c', z') with $c \neq c'$, one recovers the witness $x = (z - z')/(c - c')$.*

Proof. Subtract the verification equations to obtain $[z - z']G = [c - c']Y$. Multiplying by the inverse of $c - c'$ proves that the displayed scalar has public point Y . $\square$

For any fixed challenge c , a simulator can sample $z \leftarrow \mathbb{F}_r$ and set $A = [z]G - [c]Y$. The transcript accepts. In a real exchange with this challenge, $z = k + cx$ is uniform because translation is a bijection; the same equation determines A . Thus the simulated and real joint distributions are identical. This is special honest-verifier zero knowledge: simulation works for a supplied challenge, not merely for one sampled by the simulator. It is not yet a proof for an arbitrary verifier choosing a challenge after seeing A .

4.4 The general three-message extraction calculation

A Sigma protocol has the same first-message, challenge, response shape, perfect completeness, special soundness from distinct challenges, and special honest-verifier zero knowledge in a stated perfect, statistical or computational sense. Its challenge set C is finite; write $N = |C|$, with honest challenges uniform.

Special soundness gives an actual extractor, not just an identity conditional on somehow obtaining two transcripts. Fix the prover's complete random tape and state just before the challenge; call this a row. Let δ be the fraction of challenges for which this row accepts. Over freshly sampled rows, suppose $\mathbb{E}\delta = \epsilon$. For each extraction attempt, generate a fresh row and run it on two independent uniform challenges, restoring the saved state and random tape for the second run. The probability that both accept with distinct challenges, conditional on the row, is $\delta^2 - \delta/N$. Averaging and using nonnegative variance gives

$$\Pr[\text{successful extraction attempt}] = \mathbb{E}[\delta^2] - \epsilon/N \geq \epsilon(\epsilon - 1/N). \quad (4)$$

When $\epsilon > 1/N$, independent attempts therefore extract in expected at most $1/(\epsilon(\epsilon - 1/N))$ attempts. Each uses two prover runs and one application of special soundness. This is polynomial in the inverse excess success probability; it suffices for knowledge error $1/N$. It is not claimed to be the sharpest expected-time bound. Sampling a fresh row on every attempt prevents the extractor from remaining trapped forever on a row that accepts only one challenge.

The calculation uses classical state restoration and a fixed complete random tape. It is not automatically valid for a quantum prover, for an unreplayable external device, or for a model in which the extractor cannot restore the required state. Those are different extraction contracts.

4.5 Fiat–Shamir and the forking lemma

The Fiat–Shamir transform replaces the verifier's random challenge by a hash of an injective encoding of the statement, first message and required context. The proof contains the first message and response; the verifier recomputes the challenge. This turns interaction into a noninteractive object, but its security is not proved merely by writing down the hash expression.

The following calculation describes the classical random-oracle tool used for extraction. Suppose an algorithm makes at most $Q \geq 1$ first-time oracle queries and on success identifies one of those queries as decisive. Repeated queries reuse their earlier answer and do not receive a new index in this accounting. Let its success probability be ϵ . Run it once; if it succeeds at query index j , rewind to just before that query, keeping the algorithm's random tape and all earlier oracle answers fixed, and resample the answer at j and all later fresh answers.

Lemma 4.2 (General forking bound). *For independent uniform oracle outputs in a set of size N , the probability that both runs succeed with the same decisive index and different answers there is at least*

$$\epsilon \left(\frac{\epsilon}{Q} - \frac{1}{N} \right).$$

A negative right side is interpreted only as a vacuous lower bound.

Proof. For each index j , fix the algorithm's tape and the oracle prefix before j , and let a_j be the conditional probability of success with decisive index j . Unused query positions can be padded with dummy random answers and assigned success probability zero. The original suffix and resampled suffix are independent given this prefix, so the probability that both succeed at j , summed over j , is $\sum_j \mathbb{E}[a_j^2]$. Nonnegative variance gives $\mathbb{E}[a_j^2] \geq (\mathbb{E}a_j)^2$; applying the same inequality to the finite list of Q means gives $\sum_j (\mathbb{E}a_j)^2 \geq \epsilon^2/Q$. On an initially successful run the new answer at its decisive index equals the original with probability $1/N$. Discarding all such events costs at most ϵ/N , proving the bound. $\square$

This is the classical random-oracle lemma underlying Bellare and Neven's general forking argument. Applying it requires that the two successful outputs determine the same statement and first message at that decisive query, so that special soundness is applicable. Unqueried guesses, simulation aborts, reduction bias and additional oracle interactions have to be bounded separately.

For zero knowledge of a transformed Sigma protocol, the simulator may select a challenge, run the special honest-verifier simulator, and program that challenge at the resulting hash input. This is valid only when the point is fresh with suitably high probability and the joint distribution has the required simulation property. For example, if every first-message value has conditional probability at most 2^{-h} , then a fixed pre-existing table of at most q matching query inputs is hit with probability at most $q2^{-h}$. Here the bound is conditioned on all prior public information and the chosen challenge; an unconditional entropy statement would not suffice. The resulting simulation matches the real proof in the same perfect, statistical or computational sense, with the programming-bad probability accounted for.

4.6 Proof-system interfaces and explicit model boundaries

A noninteractive argument specifies parameter generation, proving and verification algorithms for a relation. A publicly reproducible parameter set is called transparent when no secret setup information is required. This does not prove that arbitrary public parameters are safe or that an adversary chose their encoding correctly. Succinctness is a bound on proof size and verifier work relative to the represented computation; a logarithmic proof can still have linear verification work. The actual bounds should replace the adjective whenever they matter.

In the algebraic group model, an algorithm producing a group output must also provide its expression as a linear combination of the group elements made available to it, under a specified model of oracle access. Such coefficients are additional information for an extractor. Ordinary access to an encoded point does not supply them. A security theorem proved in this model restricts the adversary accordingly; it is not a consequence of the curve equation. Likewise, composing a polynomial argument, a commitment-opening argument and Fiat–Shamir requires a theorem for that complete composition. The component definitions in this section do not establish that theorem for every construction using them.

5 RedPallas signatures

A digital signature scheme consists of key generation, a signing algorithm and a public verification algorithm. The private key authorises signing; possession of the public key permits verification but is not intended to permit creating fresh accepted signatures. We construct the exact RedPallas algorithms and then identify the security statement supplied by the Schnorr reduction.

5.1 Exact keys, signatures and verification

Use the Pallas group with scalar modulus $r = p_{\text{Vesta}}$ and a specified nonidentity base G . A private scalar x has verifying key $Y = [x]G$. Define

$$h(m) = \text{ToScalar}(\text{BLAKE2b}_{512}(\text{Zcash_RedPallasH}, m)).$$

The personalisation is exactly 16 bytes. For a message byte string M , signing performs:

1. Sample 80 independent uniform bytes T .
2. Set $k = h(T \| Y^* \| M)$, and $R = [k]G$.
3. Set $c = h(R^* \| Y^* \| M)$ and $z = k + cx$ in $\mathbb{F}_r$.
4. Return the 64 bytes $\sigma = R^* \| \text{LE}_{32}^{\text{byte}}(\text{val}(z))$.

Star encodings are packed as the 32-byte encodings in the *Math Guide*, “Pasta curves and their encodings”. The nonce is k ; its 80-byte random input must not be replaced by an undocumented deterministic or shorter construction.

Verification requires a canonically decoded public key and exactly 64 signature bytes. Decode the first 32 as a Pallas point R ; reject failure or a noncanonical encoding. Interpret the last 32 bytes little-endian as an integer z ; reject unless $z < r$. Compute $c = h(R^* || Y^* || M)$ using exactly the signature's encoded first component, then accept exactly when $[z]G = R + [c]Y$. The group cofactor is one, so there is no additional cofactor multiplication. The primitive does not reject identity R merely because it is the identity; applications may impose additional conditions on their own keys.

The algorithms implement the protocol specification's "RedDSA, RedJubjub, and RedPallas" definition, specialised to Pallas, and the `reddsa` crate's signing and verification-key operations. Their completeness is the Schnorr scalar identity in §4, independent of whether a security proof models h ideally.

5.2 Unforgeability and the simulation it needs

For the following single-key theorem, key generation samples the signing scalar uniformly from the nonzero scalars. This is a hypothesis of the unforgeability experiment, not a rule rejecting identity keys throughout the general signature interface.

In existential unforgeability under chosen-message attack, an adversary receives an honestly generated public key and access to signatures under it. A forgery is an accepted signature on a message not submitted to that oracle. Strong unforgeability also counts a different accepted signature on a previously submitted message, unless that exact signature was returned before.

In the ideal scalar-random-oracle model, a signing simulator samples independent uniform c, z , sets $R = [z]G - [c]Y$, and programs the hash at $R^* || Y^* || M$ to equal c . Before a conflicting pre-existing query occurs, this is coupled to a real Schnorr signature with a uniform nonce: the transformation $k = z - cx$ is bijective. The uniform point R hits at most q relevant old table entries with probability at most q/r . This establishes an identical-until-bad coupling, not a claim that conditioning on no conflict leaves every original marginal distribution unchanged.

For an unqueried challenge input and a nonidentity Y , a fixed purported signature satisfies the verification equation for at most one scalar challenge, so blind success has probability at most $1/r$. Otherwise a successful new signature identifies the decisive query required by Lemma 4.2. Two forks with the same encoded R, Y, M and different challenges recover x . If that input instead came from a previously simulated signature, the verification equation uniquely determines z ; canonical encodings then make the signature identical to the old one. Thus the same argument covers the strong-forgery case.

This proves a reduction with the forking loss and explicit signing-simulation error. To apply it to RedPallas's exact nonce and hash-to-scalar mechanism, make two further hybrid changes. A uniform 512-bit hash output reduced modulo r differs from a uniform scalar by at most $r/(4 \cdot 2^{512})$, from the *Math Guide*, "Probability and computation". Sum that distance over the bounded number of replaced oracle answers. The 80-byte hidden nonce prefixes are fresh except on prefix collisions or adversarial guesses; a union bound over the relevant signing and hash-query pairs gives their explicit 2^{-640} -scaled bad-event bound. The signing simulation additionally has its $1/r$ -scaled programming-conflict bound. Concretely, for at most q_s signing requests and q_h adversarial hash queries, one conservative single-execution bound is

$$\delta \leq (q_h + 2q_s + 1) \frac{r}{4 \cdot 2^{512}} + \frac{q_s(q_h + 2q_s)}{2^{640}} + \frac{q_s(q_h + q_s)}{r}.$$

The terms count scalar-distribution changes, hidden prefix coincidences, and challenge-programming conflicts, respectively. If real forgery success is ϵ , the simulated queried-forgery probability is at least $\epsilon_* = \max(0, \epsilon - \delta - 1/r)$. With $Q = \max(1, q_h)$, the resulting logarithm solver has success at least $\max(0, \epsilon_*(\epsilon_*/Q - 1/r))$, under the forking and ideal-oracle hypotheses just stated. These are distinct losses. The theorem remains a random-oracle reduction, not an unconditional proof about the fixed BLAKE2b function.

5.3 Randomised signing keys

For a scalar α , define

$$x' = x + \alpha, \quad Y' = Y + [\alpha]G.$$

The public and private transformations agree because $[x']G = [x]G + [\alpha]G$. Uniform α independent of a fixed key makes Y' uniform, proving an exact statement about the randomised public point alone. If α is revealed, the original point is recovered by subtraction; message context or other correlated data may also permit linking. Point randomisation alone is not a complete privacy theorem.

Ordinary single-key unforgeability cannot simply be renamed randomised-key security. The latter experiment allows signatures under keys $Y + [\alpha]G$, with specified adversarial choice of α , and asks for a fresh accepted signature under such a key. For the Schnorr construction above, simulate each requested effective key using its actual public point in the programmed challenge. A fork at the same decisive query fixes Y' , hence fixes α by scalar injectivity. Extraction returns

$$x = \frac{z - z'}{c - c'} - \alpha.$$

If the adversary produces an α with $Y + [\alpha]G = \mathcal{O}$, then $x = -\alpha$ already solves the original logarithm. This handles the identity effective-key case without incorrectly applying a unique-challenge argument to it. The same simulation and freshness errors still have to be included. The key prefix in the challenge is essential to fixing the effective key across forks.

The spend-authorisation base is

$$G = \text{GroupHash}_{\text{Pallas}}(z.\text{cash}:\text{Orchard}, G).$$

The binding-signature base is instead the randomness base of the relevant value commitment. These are separate generator instantiations in the protocol specification's "Spend Authorisation Signature" and "Binding Signature" sections. Using the same curve and signature equation does not make their application-level authorisation roles interchangeable.

5.4 What a binding signature contributes to balance

To expose the logical boundary, suppose commitments $C_i = [v_i]V + [\rho_i]R$ use nonidentity bases V, R , and a public signed balance is b . The point

$$B = \sum_i C_i - [b]V = \left[\sum_i v_i - b \right] V + \left[\sum_i \rho_i \right] R$$

is a public candidate verifying key under base R . When $\sum_i v_i = b$ as integers, the sum of randomness scalars is a signing key for it. Conversely, suppose a suitable composed knowledge theorem extracts both all commitment openings and a signing witness k with $B = [k]R$. Subtraction yields

$$\left[\sum_i v_i - b \right] V + \left[\sum_i \rho_i - k \right] R = \mathcal{O}.$$

If the first coefficient is nonzero modulo the group order, this is a nontrivial generator relation and breaks the binding assumption. If an independently proved range bound excludes wraparound, a nonzero integer imbalance has a nonzero field coefficient. These two steps explain why the composition can enforce integer conservation.

The hypothesis concerning joint extraction is not discharged by commitment binding, by existence of some logarithm for B , or by ordinary unforgeability for an independently generated key. An adversary may have chosen the commitments and therefore influenced B . The application's actual relation, transcript and key-generation conditions require a matching knowledge and composition theorem. The algebra above is the reduction once its witnesses are available; it does not claim that every accepted signature in every surrounding protocol yields them.

6 Key agreement and note encryption

An encryption algorithm takes a key and a plaintext byte string and produces a ciphertext. Decryption returns the original plaintext or the failure symbol $\perp$. Correctness concerns ciphertexts made honestly; confidentiality concerns what the ciphertext reveals; integrity concerns which altered ciphertexts are accepted. These are different requirements.

6.1 Diffie–Hellman and diversified bases

Fix a nonidentity point B in the prime-order Pallas group. For nonzero scalars x, e , a receiver publishes $P = [x]B$, and a sender publishes an ephemeral point $E = [e]B$. Both compute

$$S = [e]P = [ex]B = [x]E.$$

“Ephemeral” means intended for this particular encryption, not a second kind of group element. The equality proves agreement, not secrecy: computing S from B, P, E is the CDH problem defined in “Security contracts and reductions”. Nor does agreement alone authenticate a sender: anyone can choose e and encrypt to P .

The Orchard interface obtains B from `DiversifyHash( $d$ )` and uses the same nonzero incoming-viewing scalar across diversified bases. Public keys and ephemeral keys must decode canonically to nonidentity points. No cofactor multiplication is needed because the group already has prime order. Multiplication by a nonzero scalar is a bijection of its nonidentity points.

A different address base does not create a different secret scalar. Whether pairs $(B_1, [x]B_1)$ and $(B_2, [x]B_2)$ expose their common scalar is a decisional question, distinct from recovering x . Its security requires a DDH-style assumption for the actual distribution and information supplied about these bases. For example, revealing t with $B_2 = [t]B_1$ makes linkage easy: test whether the second public key equals $[t]$ times the first. The hash-to-group construction does not reveal such logarithms. This explains the role of its security model without claiming that arbitrary collections of publicly related bases are unlinkable.

Orchard derives the 32-byte encryption key by

$$K = \text{BLAKE2b}_{256}(\text{Zcash_OrchardKDF}, S^* \parallel E^*),$$

where stars denote the canonical 32-byte point encodings from the *Math Guide*, “Pasta curves and their encodings”. The ephemeral key is part of the input even though knowledge of S is the source of secrecy. This binds the derived key to the advertised ephemeral point. The concrete specification is the protocol’s Orchard key-agreement and key-derivation sections.

6.2 Authenticated encryption and nonce discipline

Authenticated encryption with associated data, abbreviated AEAD, takes a secret key K , a nonce N , associated data A and plaintext M . A nonce is a public input whose required uniqueness is specified by the scheme; it need not be secret. Associated data is authenticated but not encrypted. Encryption returns ciphertext with an authentication tag. Decryption receives the same K, N, A and either returns M or rejects.

For confidentiality, a nonce-respecting adversary supplies pairs of equal-length plaintexts and receives encryptions of one fixed hidden choice. It never repeats a nonce under the same key. Its distinguishing advantage is the difference between its output probabilities for the two choices. For ciphertext integrity, an adversary with encryption access wins by producing an accepted (N, A, C) not previously returned for that same tuple of public inputs. Variants that additionally expose decryption need to forbid trivial replay of a challenge; we do not silently import such a stronger experiment into a proof about encryption access.

Orchard uses IETF ChaCha20–Poly1305 with empty associated data and the all-zero 96-bit nonce. Consequently nonce safety rests on the derived key not being reused for different plaintexts. “The

6.3 The ChaCha20 stream, word by word

$$\begin{array}{ll} a \leftarrow a + b, & d \leftarrow \text{rotl}_{16}(d \oplus a), \\ c \leftarrow c + d, & b \leftarrow \text{rotl}_{12}(b \oplus c), \\ a \leftarrow a + b, & d \leftarrow \text{rotl}_8(d \oplus a), \\ c \leftarrow c + d, & b \leftarrow \text{rotl}_7(b \oplus c). \end{array}$$
$$(0, 4, 8, 12), \quad (1, 5, 9, 13), \quad (2, 6, 10, 14), \quad (3, 7, 11, 15), \\ (0, 5, 10, 15), \quad (1, 6, 11, 12), \quad (2, 7, 8, 13), \quad (3, 4, 9, 14).$$

6.4 Poly1305 and the authenticated byte string

```
0x0ffffffc0ffffffc0ffffffc0ffffff.
```

$$a \leftarrow (a + n)r \pmod{2^{130} - 5}.$$

22

Write $\text{pad}_{16}(X)$ for the shortest string of zero bytes that makes $|X| + |\text{pad}_{16}(X)|$ a multiple of 16. The authenticated string is exactly

$$A \parallel \text{pad}_{16}(A) \parallel C \parallel \text{pad}_{16}(C) \parallel \text{LE}_8^{\text{byte}}(|A|) \parallel \text{LE}_8^{\text{byte}}(|C|),$$

where lengths count bytes and must fit in 64 bits. The superscript specifies eight-byte encodings, as in the *Math Guide*, “Language, integers, and encodings”. The ciphertext C is the plaintext XORed with the counter-one stream. Encryption appends the resulting 16-byte tag to C .

Decryption recomputes and checks the tag before accepting any plaintext. A constant-time comparison processes the entire fixed-length tag without timing differences that depend on the first mismatching position; otherwise the implementation can expose information beyond the experiment. Empty associated data does not remove the ciphertext padding or the two length fields. All these details follow RFC 8439. We take the concrete nonce-respecting confidentiality and integrity bounds for ChaCha20–Poly1305 as cryptographic assumptions or imported analyses with their stated message limits; reproducing the permutation does not prove them.

6.5 What a hashed-DH confidentiality reduction proves

Consider a single-recipient, single-challenge experiment with independent uniform nonzero x, e , public $B, [x]B, E = [e]B$, and a random oracle H returning the encryption key on input $S^* \parallel E^*$. The adversary chooses two equal-length plaintexts; it has oracle access but no secret-key or challenge-decryption access. Replace the challenge key by an independent uniform key. The two executions can be coupled until the adversary queries $S^* \parallel E^*$.

Let the probability of that event in the independent-key experiment be α , and bound the number of oracle queries by $q_H \geq 1$. A CDH algorithm can run this experiment without knowing S , choosing its encryption key independently. It records the query candidates and outputs the first point component of a uniformly selected query slot, returning failure on a malformed or unused slot. If the critical query occurred, this outputs S with probability at least $1/q_H$. Thus $\alpha \leq q_H \text{Adv}_{\text{CDH}}$ for this constructed algorithm. Nonzero challenge scalars are part of this version of the CDH experiment.

For each of the two plaintext choices, changing the key changes an output probability by at most α for its corresponding reduction. The independent-key distinguishing advantage is exactly the confidentiality advantage of an AEAD adversary using that key. With a common upper bound on the two CDH algorithms, we obtain

$$\text{Adv}_{\text{hybrid}} \leq 2q_H \text{Adv}_{\text{CDH}} + \text{Adv}_{\text{AEAD}}.$$

The factor two belongs to switching both distinguishing experiments; confusing distinguishing advantage with excess guessing probability loses it. This theorem assumes a random-oracle KDF and fresh key conditions. It does not by itself prove recipient anonymity, chosen-ciphertext security of every surrounding note format, or security after revealing the viewing key. Authenticating ciphertext bytes likewise does not show that the decrypted fields satisfy a ledger relation.

7 Authenticated trees and append-only histories

A hash tree compresses a structured collection to a short root. An authentication path lets a verifier recompute that root from one item and a small set of siblings. The root authenticates the item only if the verifier already trusts the root or obtains it through a separate authenticated channel. Sending an item, path and arbitrary root together establishes no external truth.

7.1 Binary trees with unambiguous node encodings

A binary tree has either a leaf or an internal node with an ordered left and right child. Its height is the maximum number of edges from its root to a leaf. A perfect tree of height h has 2^h leaves at the

same depth. Encode a leaf containing bytes m by a length-delimited tuple, and hash it under a leaf tag. Encode an internal node by a different tag, its level where required, and the two fixed-length child digests in left-to-right order:

$$L(m) = H(\text{enc}(\text{leaf}, m)), \quad N_j(a, b) = H(\text{enc}(\text{node}, j, a, b)).$$

Here enc is an injective tuple encoding as constructed in the *Math Guide*, “Language, integers, and encodings”. These are generic illustrative tags, not the literal Zcash domain strings. A concrete protocol may instead use distinct personalised functions, fixed input widths, or a different level convention; the proof requires the same lack of ambiguity.

For a leaf position $i \in \{0, \dots, 2^h - 1\}$, write $i = \sum_{j=0}^{h-1} b_j 2^j$. An authentication path contains sibling digests $s_0, \dots, s_{h-1}$, ordered from leaf to root. Set $v_0 = L(m)$ and compute

$$v_{j+1} = \begin{cases} N_j(v_j, s_j) & b_j = 0, \\ N_j(s_j, v_j) & b_j = 1. \end{cases}$$

Accept if v_h equals the trusted root and all encodings, path lengths and positions are valid. There are h siblings and h internal hashes. A path proves occurrence at a specified position; it does not prove uniqueness of the underlying value.

Theorem 7.1 (Tree binding reduces to node collisions). *Fix a tree shape, node encodings and leaf positions. Two different leaf collections with the same root yield a collision in one of the tagged hash inputs. Two accepted paths for different leaf values at the same position and root do likewise.*

Proof. For full collections, compare the root inputs. If they differ while the roots agree, this is a collision. Otherwise both child digests agree; descend into a child containing a differing leaf. The finite descent either finds differing internal inputs with equal outputs or reaches differing leaf encodings with equal leaf hashes. Injective encodings ensure the last inputs really differ. For two paths, begin with the different leaf encodings. If their digests already agree there is a collision. Otherwise ascend to the first level where unequal computed values become equal. Its ordered inputs differ at least in the position occupied by the computed value, so it supplies a node collision. Direction bits must describe the same position for this argument. $\square$

A level-specific compression function may be partial, returning $\perp$ on some inputs. If a protocol maps failure to a field value, binding of the resulting total function requires analysing that failure branch too: the generic theorem does not turn two failures into a collision of a compression function whose security game excludes failing inputs. Similarly, a zero leaf is a designated value, not an absence proof unless occupancy rules make it one.

7.2 Fixed-depth append frontiers

An append operation adds the next leaf without changing earlier leaves. For capacity 2^h , define empty subtree digests recursively by $e_0 = L(\text{empty})$ and $e_{j+1} = N_j(e_j, e_j)$, with the concrete empty leaf fixed by the protocol. A frontier stores one completed subtree at each height where the binary expansion of the current leaf count n has a one. The stored subtrees partition the occupied prefix in left-to-right order of decreasing heights.

To append a new leaf, set a carry to its digest at height zero. While a frontier entry exists at the carry’s height, hash that old entry on the left and the carry on the right, erase the entry, and raise the carry’s height by one. Store the carry at the first empty height, then increment n . This is exactly binary addition with hash operations in place of numerical carries. The invariant about the occupied prefix follows by induction: two adjacent completed subtrees of equal size are replaced by their parent and nothing earlier changes.

When $n < 2^h$, the full padded root can be recovered with $v = e_0$. For $j = 0, \dots, h - 1$, set

$$v \leftarrow \begin{cases} N_j(f_j, v) & \text{bit } j \text{ of } n \text{ is one,} \\ N_j(v, e_j) & \text{bit } j \text{ of } n \text{ is zero.} \end{cases}$$

Here f_j is the occupied frontier entry at height j . At each step, v is the partially occupied or empty right-hand region needed at that height. For $n = 2^h$, the frontier entry at height h is already the full root; appending again is invalid. The count is needed: a frontier is not simply an unordered collection of digests.

An individual append may merge h times, but over n appends fewer than n merges occur. At height $j \geq 1$, there are at most $\lfloor n/2^j \rfloor$ newly formed parents; their sum is less than n . This amortised bound means a bound on total work divided by operation count, not a worst-case bound for every operation. Recovering a padded root separately still costs h hashes in this simple construction.

7.3 Peaks, history commitments and structural binding

An unbounded append history can retain the same completed subtrees without padding to a fixed capacity. Its current forest is an ordered list of peaks, one for each nonzero bit in the leaf count; a peak is the root of one completed subtree. This construction is commonly called a Merkle mountain range. The binary carry rule determines its shape exactly from the leaf count. For example, thirteen leaves partition as $8 + 4 + 1$, so the peak heights are 3, 2, 0 in that order.

To obtain a single commitment, specify a function that combines the ordered peaks and authenticates the information needed to recover their shape. One simple generic choice is a separate tagged hash of the count and a length-delimited list of (height, digest) pairs. Other protocols combine peaks recursively. These choices are not interchangeable on the wire. An inclusion proof consists of a path inside one peak and the data needed to authenticate that peak in the committed ordered forest.

A history node can also authenticate metadata, such as a covered interval or aggregate work. For example, an encoding may include the two child digests and their claimed aggregate values. Hashing this tuple binds the claim as bytes; it does not prove that the aggregate equals the sum of valid child work. The verifier must check the combination rule and the leaf rules where its security theorem requires them. This distinction is essential when a proof opens only sampled parts of a large history.

An append-consistency proof establishes that one committed sequence is a prefix of another. For a direct construction, supply the earlier frontier and the appended leaves, authenticate the earlier root, then replay the carry rule and compare the later root. Tree binding makes two inconsistent reconstructions a hash-collision event, provided counts and shapes are bound. This direct proof can be large. A smaller proof may replace groups of new leaves by completed subtree roots, but then its exact decomposition, ordering and endpoint checks must be specified. Membership alone is not append consistency.

The generic constructions here discharge the hash-tree and append invariants. Exact ledger node encodings, history metadata, peak-combining rules and block commitment composition are separate protocol choices; their security inherits these lemmas only after those choices satisfy the hypotheses. No network activation claim follows from the name of a tree algorithm.

8 Wallet permutations and address encodings

A wallet needs more than a secure scalar generator. It must map indices to diversifiers without repetition, encode structured receiver data, and detect damaged text. This section constructs the lower-level transformations. A secret-key permutation, a public reversible mixing function, and an error-detecting checksum have different contracts even if all three change bytes.

8.1 AES-256 from the byte field

A block cipher is a keyed family of permutations of fixed-length blocks. AES-256 has a 32-byte key and 16-byte blocks. The number 256 describes the key length, not the block length. Its byte arithmetic uses exactly the field constructed in the *Math Guide*, “Byte fields for AES”: XOR addition and polynomial multiplication reduced by $X^8 + X^4 + X^3 + X + 1$. We denote multiplication in this field by a centred dot.

AES begins with a substitution of each byte. For input a , let $u = a^{-1}$ in the byte field when $a \neq 0$, and let $u = 0$ when $a = 0$. Write u_i for bit i , counting from the least significant bit. Output bit i is

$$u_i \oplus u_{(i+4) \bmod 8} \oplus u_{(i+5) \bmod 8} \oplus u_{(i+6) \bmod 8} \oplus u_{(i+7) \bmod 8} \oplus c_i,$$

where $c = 0x63$. This defines the entire substitution table, conventionally called the S-box, without importing 256 unexplained entries. The inversion map, with zero fixed, is a permutation. The following inverse for the affine bit map shows that the S-box is a permutation too: from output bits v_i , recover

$$u_i = v_{(i+2) \bmod 8} \oplus v_{(i+5) \bmod 8} \oplus v_{(i+7) \bmod 8} \oplus d_i, \quad d = 0x05.$$

Substitute the preceding formula and cancel each duplicated bit to verify this identity. For example, $0x53^{-1} = 0xca$ and its substituted value is $0xed$; zero maps to $0x63$.

Arrange an input block as a 4-by-4 byte array $s_{r,c}$ with $s_{r,c} = \text{input}[4c + r]$. The following transformations operate on that array:

- SubBytes replaces every byte by its S-box value.
- ShiftRows sends the old $s_{r,(c+r) \bmod 4}$ to position (r, c) : row r rotates left by r .
- MixColumns replaces each column vector by its product with the byte-field matrix

$$\begin{pmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{pmatrix}.$$

The entries are hexadecimal byte values, not decimal scalars acting by repeated addition in characteristic two.

- AddRoundKey XORs a 16-byte round key into the state in the same column-major order.

The inverse MixColumns matrix is

$$\begin{pmatrix} 0e & 0b & 0d & 09 \\ 09 & 0e & 0b & 0d \\ 0d & 09 & 0e & 0b \\ 0b & 0d & 09 & 0e \end{pmatrix}.$$

Multiplying the two matrices in the byte field gives the identity, a direct verification rather than an assumption. ShiftRows is reversed by rotating right, and AddRoundKey is its own inverse. Every round is therefore invertible.

To derive round keys, divide the 32-byte key in order into eight four-byte words $w_0, \dots, w_7$. A word here is an ordered four-byte tuple; no host integer endianness is implicit. Define $\text{RotWord}(a, b, c, d) = (b, c, d, a)$ and apply the S-box to each byte for SubWord. Generate w_i for $i = 8, \dots, 59$ as follows. Begin with $t = w_{i-1}$. If $i \bmod 8 = 0$, replace

$$t \leftarrow \text{SubWord}(\text{RotWord}(t)) \oplus (\rho_{i/8}, 0, 0, 0),$$

where $\rho_1 = 01$ and $\rho_{j+1} = 02 \cdot \rho_j$ in the byte field. If $i \bmod 8 = 4$, replace $t \leftarrow \text{SubWord}(t)$ instead. In all cases set $w_i = w_{i-8} \oplus t$. Round key j , for $0 \leq j \leq 14$, is the concatenation $w_{4j} \parallel \dots \parallel w_{4j+3}$.

Encryption first applies AddRoundKey with key zero. Rounds 1 through 13 apply SubBytes, ShiftRows, MixColumns, and AddRoundKey with that round's key, in that order. Round 14 omits MixColumns. Serialise the resulting array in its original column-major order. Decryption is well-defined by reversing this complete sequence and applying each inverse transformation. FF1 below only calls AES encryption, even when decrypting its own format-preserving permutation.

These are the AES-256 algorithms of FIPS 197. Their invertibility is proved by the component inverses. Their resemblance to a random permutation under an unknown uniformly sampled key is the AES PRP assumption, not a consequence of invertibility or of having 14 rounds.

8.2 The exact 88-bit FF1 permutation

A format-preserving permutation maps a prescribed finite set bijectively to itself. The diversifier permutation uses AES-256-based FF1 on bit strings of length 88, with radix two and an empty tweak. Radix means the number of possible numeral values; here each numeral is a bit. A tweak is an additional public parameter selecting a permutation under the same secret key; this protocol fixes it to the empty byte string. We specialise the algorithm to these parameters, so no unused decimal or variable-length cases are needed.

For a bit string $B = (b_0, \dots, b_{43})$, define

$$\text{NUM}_2(B) = \sum_{j=0}^{43} b_j 2^{43-j}.$$

Its inverse $\text{STR}_{2,44}$ writes a number in $[0, 2^{44})$ as exactly 44 bits, most significant first. This numeral interpretation is distinct from the wallet's little-endian encoding of an 88-bit index. If the index is j , the FF1 input sequence is its 88 little-endian bits; the first bit of each 44-bit half then becomes the most significant numeral of that half inside FF1. There is no contradiction: these are two explicitly different maps applied consecutively.

Let the secret key be K , and split the 88-bit input into consecutive halves A, B of 44 bits. Construct the fixed 16-byte block

$$P = [1, 2, 1, 0, 0, 2, 10, 44] \parallel \text{BE}_4^{\text{byte}}(88) \parallel \text{BE}_4^{\text{byte}}(0).$$

For $i = 0, \dots, 9$, form

$$Q_i = 0^9 \parallel [i] \parallel \text{BE}_6^{\text{byte}}(\text{NUM}_2(B)).$$

Here 0^9 means nine zero bytes, and $[i]$ one byte; Q_i has exactly 16 bytes. The two-block chaining computation is

$$R_i = \text{AES}_K(\text{AES}_K(P) \oplus Q_i).$$

Interpret the first 12 bytes of R_i as a big-endian integer y_i , and update

$$C = \text{STR}_{2,44}((\text{NUM}_2(A) + y_i) \bmod 2^{44}), \quad (A, B) \leftarrow (B, C).$$

After ten rounds, output $A \parallel B$. To represent this output as 11 bytes, put sequence bit $8j + k$ in bit k of byte j , matching the initial little-endian bit-sequence convention.

For connection with the general FF1 specification, the half-lengths are $u = v = 44$, the byte length of a half-numeral is $b = \lceil 44/8 \rceil = 6$, and the output expansion length is $d = 4\lceil b/4 \rceil + 4 = 12$. Since $d < 16$, the expansion uses only the first block's prefix; there are no omitted extra AES blocks in this case. The fixed prefix encodes these parameters and the zero tweak length. This is the specialisation of NIST SP 800-38G used by the protocol's diversifier PRP.

Proposition 8.1 (FF1 is a permutation). *For every fixed key, the ten-round function above is a bijection of the 2^{88} input bit strings.*

Proof. Write the round's derived integer as $F_i(B)$. From output (A', B') , the prior right half is $B = A'$. Recompute $F_i(A')$ and recover the prior left integer as $(\text{NUM}_2(B') - F_i(A')) \bmod 2^{44}$. Thus every round has a unique inverse. Apply these inverses for $i = 9, 8, \dots, 0$ to invert the whole construction. This argument does not assume that the round function itself is invertible. $\square$

Distinct indices therefore produce distinct 88-bit diversifier strings under one key. Their distribution as a secret-key pseudorandom permutation is a separate FF1 security contract, using its analysis and AES assumption for this domain and query count. A permutation over 2^{88} elements cannot provide an unlimited 256-bit domain-size security claim merely because its AES key has 256 bits. Nor does permutation injectivity assert that a subsequent hash-to-group map is injective.

8.3 F4Jumble: a public reversible mixing function

F4Jumble accepts a byte string of length $48 \leq n \leq 4\,194\,368$. Split it into $L||R$ with $\ell = \min(64, \lfloor n/2 \rfloor)$ bytes on the left and $r = n - \ell$ on the right. For $i \in \{0, 1\}$, define

$$H_i(R) = \text{BLAKE2b}_{8\ell}(\text{UA_F4Jumble_H}||[i, 0, 0], R),$$

$$G_i(L) = \left(\bigparallel_{j=0}^{\lceil r/64 \rceil - 1} \text{BLAKE2b}_{512}(\text{UA_F4Jumble_G}||[i]|| \text{LE}_2^{\text{byte}}(j), L) \right) [0 : r].$$

The strings before the appended bytes have length 13, so each personalisation is exactly 16 bytes. The maximum right length is $2^{16}64$, so its last block index fits in two bytes. The notation $[0 : r]$ means the first r bytes, not a field reduction. Perform four updates in order:

$$R \leftarrow R \oplus G_0(L), \quad L \leftarrow L \oplus H_0(R), \quad R \leftarrow R \oplus G_1(L), \quad L \leftarrow L \oplus H_1(R).$$

Concatenate the final L, R . To invert, perform the same updates in reverse order, beginning with $L \leftarrow L \oplus H_1(R)$. Each update changes one half by XOR with a function of the unchanged half, and so is its own inverse at that stage. This proves bijectivity at every valid length.

There is no secret key. Anyone can invert the function or deliberately construct a chosen output. It therefore supplies neither encryption nor message authentication. Its role is to spread changes across an encoded structured payload before text encoding and checksum computation. Claims about resisting specially structured mutations need the corresponding analysis, not just the four-round invertibility proof. The construction and address application are specified in ZIP 316.

8.4 Bech32m: symbols, checksum and canonical decoding

An error-detecting checksum is redundant data computed publicly from a message to help detect accidental damage. Because anyone can recompute it, a valid checksum is not evidence that an adversary did not replace the message. Bech32m represents values 0 through 31 using this ordered alphabet:

qpzry9x8gf2tvdw0s3jn54khce6mua7l.

To convert a byte payload to these five-bit symbols, read each byte most significant bit first, concatenate the bits, split into groups of five, and pad the final group with zero bits on the right. There are $\lceil 8n/5 \rceil$ symbols for n bytes. Inverting this operation must reject five or more leftover bits and must reject nonzero leftover bits. Otherwise multiple symbol strings could encode the same bytes. This bit convention differs from the little-endian diversifier-bit packing above.

The human-readable part, abbreviated HRP, is a nonempty string of at most 83 printable ASCII characters with codes 33 through 126. It identifies the application or network namespace. For checksum calculation use lowercase and expand it into the integer sequence

$$(\lfloor c_0/32 \rfloor, \dots, \lfloor c_{h-1}/32 \rfloor, 0, c_0 \bmod 32, \dots, c_{h-1} \bmod 32),$$

where c_i are its ASCII codes. Define five 30-bit constants, written in hexadecimal, by

$$\begin{aligned} g_0 &= 3b6a57b2, & g_1 &= 26508e6d, & g_2 &= 1ea119fa, \\ g_3 &= 3d4233dd, & g_4 &= 2a1462b3. \end{aligned}$$

Starting with $z = 1$, process each input symbol $v \in [0, 31]$ by first setting $b = \lfloor z/2^{25} \rfloor$ and then

$$z \leftarrow ((z \bmod 2^{25})2^5) \oplus v \oplus \bigoplus_{i: \text{bit } i \text{ of } b=1} g_i.$$

Call the final value `polymod` of the symbol sequence. This recurrence is a complete algorithm; using the name does not require any unstated coding theory.

For expanded HRP E and payload symbols D , compute

$$z = \text{polymod}(E\|D\|[0, 0, 0, 0, 0, 0]) \oplus 0x2bc830a3.$$

Append the six five-bit digits of z , most significant first, to D . The encoded text is HRP, the character 1, and the alphabet characters for these data and checksum symbols. The verification condition is

$$\text{polymod}(E\|D\|\text{checksum}) = 0x2bc830a3.$$

For example, `a11qfn3a` is the valid empty-payload Bech32m string with HRP `a`. The older Bech32 checksum uses constant one instead; accepting that constant would accept a different format, not a second spelling of Bech32m.

A decoder rejects mixed letter case, invalid ASCII or alphabet characters, an empty HRP, missing separator or fewer than six checksum symbols. The separator is the last 1, since this character can occur in the HRP but not in the data alphabet. Uniform uppercase can be normalised to lowercase for checksum calculation when the application permits it; canonical encoders choose a single case. After checksum verification, remove the six checksum symbols, invert the bit conversion with its padding checks, then enforce the application's exact HRP, payload length and structural decoding rules.

The original generic Bech32 format limits the whole text to 90 characters. Unified address encodings explicitly remove that limit, so a generic library's 90-character rejection cannot be imported unchanged. Conversely, length-specific error-detection guarantees cannot automatically be extended to every longer unified address. The checksum and generic format are defined by BIP 350 and BIP 173; the Zcash length and payload rules are in ZIP 316. The recurrence is fully specified here; the exhaustive classification of detected error patterns is an imported coding result, not a cryptographic authenticity theorem.

8.5 The composition boundary

The roles now separate cleanly. AES supplies a keyed block permutation. FF1 uses it to permute the index domain without repetitions. Hash-to-group turns the resulting diversifier into a public curve base. F4Jumble reversibly mixes a structured byte payload. Bech32m converts those bytes into checksummed text. Each transformation has explicit input sizes, bit order and failure conditions.

No individual primitive proves the validity of a receiver tuple or secret-key derivation. That requires the application's tuple encoding, domain tags, key relationships and validation rules. In particular, neither a valid Bech32m checksum nor successful F4Jumble inversion establishes ownership of the advertised receiver. Their job is faithful representation, not authority.